

CPE 486/586: Deep Learning for Engineering Applications

10 Physics-informed Neural Networks

Spring 2026

Rahul Bhadani

Outline

1. Motivation

Motivation

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is $x = e^{-2t}$ with initial condition $x(0) = 1$.

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is $x = e^{-2t}$ with initial condition $x(0) = 1$.

But it is usually not analytically possible to solve all kind of differential equations.

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t , we know x and we want to find f such that it can give x for any value of t .

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t , we know x and we want to find f such that it can give x for any value of t .

What's the problem then?

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t , we know x and we want to find f such that it can give x for any value of t .

What's the problem then?

It may not necessarily be the solution to the differential equation provided earlier!

Differential Equation as Regularizer

Let's rewrite our differential equation

Consider a differential equation:

$$\frac{dx(t)}{dt} + 2x = 0$$

But as the real world data will exhibit some error, $x(t)$ might not satisfy the above equation. But if we approximate $x(t)$ using some neural network $u^\theta(x, t)$, we would want

$$\frac{du^\theta(x, t)}{dt} + 2u^\theta(x, t) \leq \varepsilon$$

where ε

Differential Equation as Regularizer

Hence, apart from the training loss, we introduce a physics loss

Consider a differential equation:

$$\mathcal{L}_{physics}$$

which is

$$\mathcal{L}_{physics} = \frac{du^{\theta}(x, t)}{dt} + 2u^{\theta}(x, t)$$

for the previous example that we would be looking to minimize.

This is what we call **Physics-Informed Deep Learning (PIDL)** or in general **Physics Informed Machine Learning (PIML)** and the Neural Network used would be called **Physics-Informed Neural Network (PINN)**.

Updated Loss Function

$$\mathcal{L} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{physics} + \lambda_3 \mathcal{L}_{IC} + \lambda_4 \mathcal{L}_{BC}$$

where IC is for initial conditions and BC is for boundary conditions.

Physics governs almost every system we want to model.

Classical numerical solvers (FEM, FDM, FVM):

- ⚡ Require fine meshes
- ⚡ Scale poorly to high dimensions
- ⚡ Cannot easily solve inverse problems
- ⚡ Need full boundary conditions

Pure data-driven ML:

- ⚡ Needs enormous labelled data
- ⚡ No physics guarantee
- ⚡ Fails to extrapolate
- ⚡ Black-box predictions

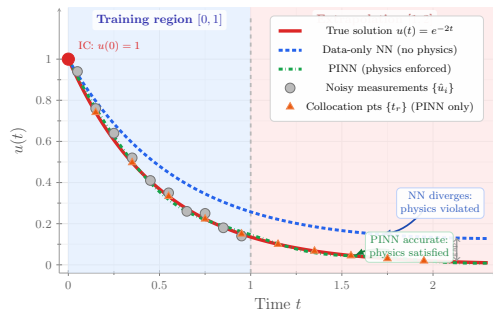
Can we get the best of both worlds?

Data Alone is Not Enough

Suppose we fit a neural network $u^\theta(x, t)$ purely on noisy measurements $\{(t_i, \hat{x}_i)\}$.

Problems:

- ⚡ The network may **overfit** to noise
- ⚡ **Extrapolation** outside training region fails
- ⚡ The prediction is **not guaranteed** to satisfy $\frac{dx}{dt} = -2x$
- ⚡ Different networks with the same data error can give **physically inconsistent** results



Red: truth
PINN

Blue: data-only NN

Green:

The Complete PINN Loss

We want $u^\theta(x, t)$ to simultaneously:

- 1 **Fit the data** at measurement points $\{t_i, \hat{x}_i\}$
- 2 **Satisfy the ODE** at collocation points $\{t_r\}$
- 3 **Satisfy the initial condition** $u^\theta(x, 0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d} \sum_{i=1}^{N_d} |u^\theta(x, t_i) - \hat{x}_i|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{du^\theta(x, t_j)}{dt} \right|_{t_j} + 2u^\theta(x, t_j)}_{\mathcal{L}_{\text{physics}}}^2 + \lambda_0 \underbrace{|u^\theta(x, 0) - 1|^2}_{\mathcal{L}_{\text{ic}}}$$

The Complete PINN Loss

We want $u^\theta(x, t)$ to simultaneously:

- 1 **Fit the data** at measurement points $\{t_i, \hat{x}_i\}$
- 2 **Satisfy the ODE** at collocation points $\{t_r\}$
- 3 **Satisfy the initial condition** $u^\theta(x, 0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d} \sum_{i=1}^{N_d} |u^\theta(x, t_i) - \hat{x}_i|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{du^\theta(x, t_j)}{dt} \right|_{t_j} + 2u^\theta(x, t_j)}_{\mathcal{L}_{\text{physics}}}^2 + \lambda_0 \underbrace{|u^\theta(x, 0) - 1|^2}_{\mathcal{L}_{\text{ic}}}$$

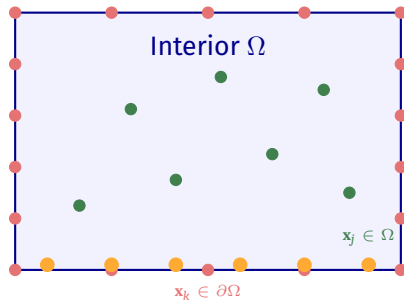
The physics loss needs **derivatives of the network output** — these are computed *exactly* via **automatic differentiation**.

The Complete PINN Loss: General Case

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\frac{\lambda_d}{N_d} \sum_{i=1}^{N_d} \left| u^\theta(\mathbf{x}_i, t_i) - \hat{u}_i \right|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\frac{\lambda_r}{N_r} \sum_{j=1}^{N_r} \left| \Psi[u^\theta](\mathbf{x}_j, t_j) - f(\mathbf{x}_j, t_j) \right|^2}_{\mathcal{L}_{\text{physics}}} \\ + \underbrace{\frac{\lambda_b}{N_b} \sum_{k=1}^{N_b} \left| \mathcal{B}[u^\theta](\mathbf{x}_k, t_k) - g(\mathbf{x}_k, t_k) \right|^2}_{\mathcal{L}_{\text{BC}}} + \underbrace{\frac{\lambda_0}{N_0} \sum_{m=1}^{N_0} \left| u^\theta(\mathbf{x}_m, 0) - u_0(\mathbf{x}_m) \right|^2}_{\mathcal{L}_{\text{IC}}}$$

Symbol	Meaning	Evaluated on
$\Psi[\cdot]$	Differential operator (PDE/ODE)	Interior $\Omega \times (0, T]$
$\mathcal{B}[\cdot]$	Boundary operator (Dirichlet / Neumann / Robin)	$\partial\Omega \times (0, T]$
$f(\mathbf{x}, t)$	Forcing / source term	
$g(\mathbf{x}, t)$	Prescribed boundary value	
$u_0(\mathbf{x})$	Initial condition	$\Omega \times \{t = 0\}$
$\lambda_d, \lambda_r, \lambda_b, \lambda_0$	Loss weights (hyperparameters)	

Sampling Sets for Each Loss Term



- Collocation (physics)
- Boundary (BC)
- Initial time $t = 0$ (IC)
- Measurement (data, if available)

None of these sets require a mesh — points can be placed **anywhere**, including adaptively where the residual is largest.

Two Modes: Forward and Inverse

Forward Problem

Given: The PDE + BCs/ICs

Find: The solution $u(x, t)$

Physics $\longrightarrow$ Solution

Inverse Problem

Given: Sparse measurements of u

Find: Unknown PDE parameters γ

Measurements $\longrightarrow$ Physics

Two Modes: Forward and Inverse

Forward Problem

Given: The PDE + BCs/ICs

Find: The solution $u(x, t)$

Physics $\longrightarrow$ Solution

Inverse Problem

Given: Sparse measurements of u

Find: Unknown PDE parameters γ

Measurements $\longrightarrow$ Physics

PINNs handle both in the same framework
(just add unknown parameters λ as trainable variables)

References

- ⚡ Physics-informed machine learning <https://doi.org/10.1038/s42254-021-00314-5>
- ⚡ Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations <https://arxiv.org/abs/1711.10561>
- ⚡ Physics Informed Deep Learning (Part II): Data-driven Discovery of Nonlinear Partial Differential Equations <https://arxiv.org/abs/1711.10566>
- ⚡ Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations Author links open overlay panel <https://doi.org/10.1016/j.jcp.2018.10.045>
- ⚡ <https://github.com/maziarraissi/PINNs>
- ⚡ <https://github.com/prateekbhustali/Physics-Informed-Neural-Networks>
- ⚡ https://github.com/nguyenkhoa0209/pinns_tutorial
- ⚡ <https://annien094.github.io/PINNs-tutorial-MICCAI-2024/>
- ⚡ Physics-informed neural networks for PDE problems: a comprehensive review <https://doi.org/10.1023/a:1012784129883>
- ⚡ A hands-on introduction to Physics-Informed Neural Networks for solving partial differential equations with benchmark tests taken from astrophysics and plasma physics <https://arxiv.org/pdf/2403.00599>